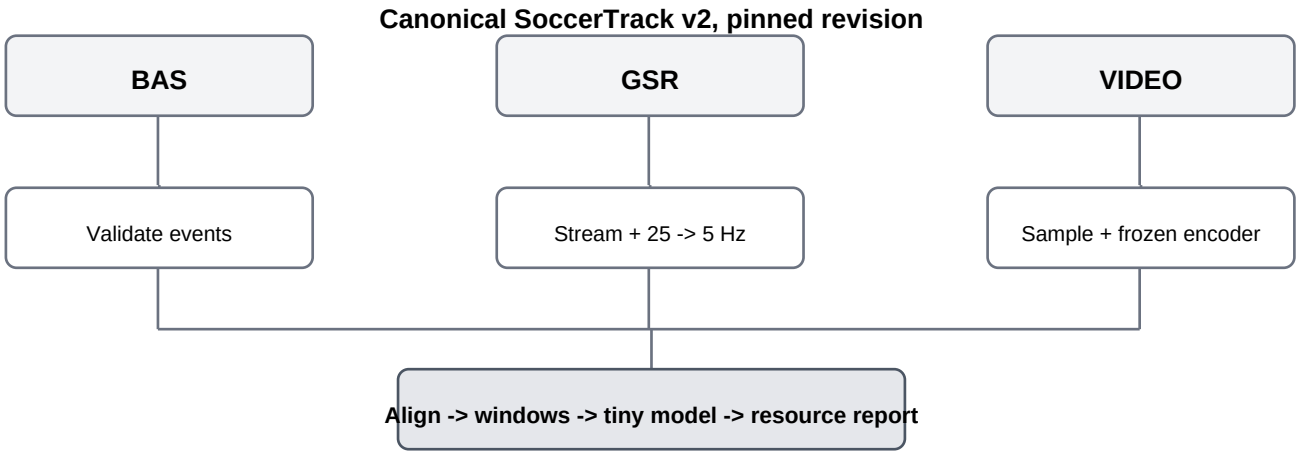
Feasibility Study Guide

Football Ball Action Anticipation with SoccerTrack v2

Goal: run a small, repeatable end-to-end pilot that proves the dataset can be versioned, streamed, aligned, compressed, trained on, and projected to full-scale work within the team's Colab constraints. This is a technical feasibility test, not a scientific accuracy experiment.



Pilot definition

Item	Pilot choice
Match	117093, unless the pinned canonical revision reveals a new issue
Segment	First valid 10 minutes of one half
Inputs	BAS + GSR + matching panoramic video
Pilot context	Past 5 s -> predict next 5 s
GSR starting rate	Downsample 25 Hz -> 5 Hz
Video starting rate	About 6.25 fps, frozen visual encoder
Compute	CPU for parsing/alignment; GPU only for visual extraction and tiny training
Do not use	Match 132831 until corrected GSR is confirmed

Overall pass condition: the pilot reaches a valid future-action output, all provenance and alignment gates pass, the tiny model can overfit a very small batch, and the measured full-scale projection leaves meaningful resource headroom.

Step 0 - Read the handoff and freeze the rules

Before opening Colab, every teammate should read **context.md**. Do not independently redefine the task, choose a different match, or use a different dataset mirror without recording the reason.

> **Validator: confirm that the operator understands that the pilot tests technical feasibility only. It cannot prove that game state improves BAA.**

Step 1 - Create a reproducible environment

Use a fresh Colab notebook. Start on a standard CPU runtime. Record Python version, package versions, date, and repository commit. Clone the official SoccerTrack v2 repository. Do not request a GPU yet.

```
git clone https://github.com/AtomScott/SoccerTrack-v2.git
cd SoccerTrack-v2
git rev-parse HEAD
```

Output: `pilot_manifest.json` containing repository commit, data source/revision, match ID, segment, sampling rates, and environment information.

> **PASS: another teammate can identify the exact code and data version used.**

Step 2 - Pin the canonical data source

Use the canonical Hugging Face SoccerTrack v2 distribution. The official project page states that mirrors may lag. The Google Drive folder is useful but must not silently replace the canonical revision for research experiments.

Canonical dataset: https://huggingface.co/datasets/atomscott/soccertrack-v2

Project page: https://atomscott.github.io/SoccerTrack-v2/

Record: filenames, sizes, hashes, download source, and dataset revision if exposed by the downloader.

> **PASS: BAS, one GSR half, and matching video for match 117093 are obtainable and versioned.**

Step 3 - Validate BAS before touching the huge files

BAS is small. Inspect it first. Confirm the top-level schema, class names, half/timestamp representation, player ID availability, and the exact events inside the chosen 10-minute segment. Prefer the repository loader or documented time helper rather than manually assuming that second-half `position` is half-relative.

BAS docs: current BAS format

Output: `pilot_events.parquet` and `bas_audit.json`. Required columns include canonical event time, class, team, player_id when present, and source record index.

PASS: all retained pilot events lie inside the chosen segment, use recognized classes, and have an unambiguous time mapping.

Step 4 - Stream GSR, do not json.load the whole half

Current SoccerTrack developer notes warn that shipped GSR differs from the older simplified schema. A half can be about 2.7 GB and naive loading can require roughly 20 GB of RAM. Parse the actual shipped structure incrementally or use the repository loader. Keep only the information required for this thesis.

GSR docs and release warning: current GSR format

```
Keep: time/frame, trajectory key, player_id metadata, role, team, x, y
Derive: vx, vy
Discard for pilot model: unnecessary image boxes, calibration payload, unrelated metadata
Sample: 25 Hz -> 5 Hz
```

Output: `pilot_state.npz` with a fixed player-slot tensor and validity mask. Keep identity fields in preprocessing metadata, not as learned semantic inputs.

PASS: the 10-minute state segment is produced without holding the full multi-GB GSR file in RAM.

Step 5 - Validate GSR spatially

Plot player positions on a standard 105 m x 68 m pitch for at least 10 timestamps. Compare several of those times with corresponding panoramic video frames. The question is not perfect pixel correspondence. The question is whether positions, sides, and timing are physically plausible.

PASS: no systematic spatial distortion, impossible player distribution, or unexplained time shift is observed. If a systematic problem appears, stop before model work.

Step 6 - Sample the video and extract frozen features once

Use the same 10-minute interval. Start near 6.25 fps and resize before inference. Use one lightweight pretrained visual encoder in evaluation mode with gradients disabled. The feasibility question is whether video can become a compact feature sequence at acceptable cost, not which backbone is best.

```
10 minutes x 6.25 fps ~= 3,750 sampled frames
4K video -> sampling/resize -> frozen encoder -> FP16 embeddings -> pilot_visual.npy
```

Measure: wall-clock time, accelerator type, frames/second, peak RAM, peak VRAM, input size, and output feature size.

> **PASS: feature extraction completes, outputs are compact, and no end-to-end backbone training is needed.**

Step 7 - Put BAS, GSR, and video on one time axis

Create one canonical pilot timeline. For each selected time, associate the correct state tensor, nearest/appropriate visual feature, and future BAS targets. Never infer alignment from file order alone.

For second-half work later, use the repository-supported half-relative time conversion. The current developer docs explicitly warn that naive use of BAS `position` is wrong for half 2 in shipped files.

> **Manual validator: inspect about 20 randomly chosen windows. Confirm past-video boundary, future-video boundary, event labels, GSR state, and that no future information enters the input.**

Step 8 - Build simple pilot windows

For feasibility only, use 5 seconds of past context and predict all events in the next 5 seconds, with a 5-second stride. Keep empty future windows. Do not create windows by centering on known future events.

```
window record:
  match_id
  half
  context_start, context_end
  future_start, future_end
  visual slice reference
  state slice reference
  target events: [(class, future_offset), ...]
```

Output: `pilot_windows.parquet`.

> **PASS: expected window count is reproducible, empty/multi-event windows are handled, and no window crosses a half boundary.**

Step 9 - Run the deterministic validator before training

Check	Expected result
Provenance	repo commit + data revision/source + hashes recorded
BAS	valid classes and canonical times
GSR	streamed successfully; compact tensor shape fixed
Spatial	pitch plots plausible
Temporal	manual BAS/GSR/video checks pass
Windows	no future leakage; no cross-half windows
Storage	processed outputs much smaller than raw inputs

STOP rule: do not train if provenance, temporal alignment, or future-leakage checks fail.

Step 10 - Overfit 16 to 32 windows

Use a tiny simple-fusion network. For example: a small GRU for the visual sequence, a small GRU for the flat state sequence, concatenate their summaries, and predict future event class/time with a very small head. Do not build the full graph model yet.

Intentionally train on only 16 to 32 windows. The goal is to see the training loss fall strongly and to confirm that labels, time normalization, matching, checkpointing, and inference all work.

PASS: the tiny batch can be overfit, no NaNs occur, a checkpoint can save/load, and sample class/time predictions are produced.

Step 11 - Run the complete 10-minute pilot

After the overfit sanity check passes, run the same tiny model over the full pilot windows. Do not interpret the resulting accuracy as scientific evidence because the pilot is one small contiguous segment from one match.

Outputs: `pilot_model.pt`, `pilot_training_log.csv`, `pilot_predictions.json`.

Step 12 - Measure resource use and project full-scale cost

Metric	Record
Raw segment duration	10 min
Video input size	...
GSR input size	...
BAS size	...
GSR parse time / peak RAM	...
Visual extraction time / peak VRAM	...
State output size	...
Visual output size	...
Pilot windows	...
Tiny model params / epoch time	...
Accelerator/compute use if measurable	...

For approximately linear stages such as frozen visual extraction, the 10-minute pilot represents about 1/90 of the roughly 900-minute dataset. Use measured cost x 90 as a first projection, then add a safety margin for I/O variability and reruns. Do not assume neural-network training scales in exactly the same way.

Budget rule: if the team later has about 100 paid compute units, prefer a projection where mandatory work fits within roughly 70-80 units, preserving 20-30 units for failures, reruns, and essential ablations. This is an internal planning threshold, not an official Colab conversion.

Step 13 - Independent teammate replication

A second teammate should repeat the pilot independently with the same pinned revision, match, segment, rates, and validation rules. They should not copy the first run's intermediate files.

Must match exactly	May differ
retained BAS event count	wall-clock time
state tensor shape	peak VRAM/RAM by small amounts
visual embedding shape	accelerator model if free Colab differs
window count	throughput
class/time alignment logic	minor numeric training trajectory

Validator: if independent runs disagree on event counts, tensor shapes, windows, or time alignment, the pipeline is not yet validated.

Step 14 - Issue a GO / MODIFY / NO-GO decision

Decision	Use when	Typical next action
GO	All critical gates pass; resource projection has headroom	Proceed to one-fold B1-B5 research pipeline
MODIFY	Concept works but one design choice is too costly or fragile	Lower sampling rate, smaller backbone, fix alignment, quarantine bad ma
NO-GO	Core data/alignment/access or resource requirement remains infeasible after reasonable changes	Stop before spending budget

Required feasibility report

Each teammate should submit a short report with exactly these sections:

```
1. Environment and pinned revisions
2. Pilot data used
3. BAS validation result
4. GSR streaming and compression result
5. Video feature-extraction result
6. BAS/GSR/video alignment checks
7. Pilot window statistics
8. Tiny-model sanity result
9. Resource measurements and full-scale projection
10. Problems encountered and corrections
11. Final decision: GO / MODIFY / NO-GO
```

Attach or preserve the generated artifacts: manifest, audits, compact features, window table, training log, sample predictions, and resource log. Do not hide failed checks. A failure discovered during feasibility is useful because it prevents expensive full-scale mistakes.

Known issue to remember: match 132831

The current SoccerTrack repository documents two swapped calibration keypoints for match 132831. The as-shipped calibration RMS is reported as 1260.95 px versus 18.07 px after correction, and the current correction note says the shared GSR labels still need regeneration. Therefore 132831 is excluded from this pilot and remains quarantined for full experiments until corrected GSR is confirmed.

Correction notes: https://github.com/AtomScott/SoccerTrack-v2/tree/main/data_corrections

Reference links

SoccerTrack v2 project: https://atomscott.github.io/SoccerTrack-v2/

SoccerTrack v2 GitHub: https://github.com/AtomScott/SoccerTrack-v2

Canonical Hugging Face dataset: https://huggingface.co/datasets/atomscott/soccertrack-v2

Google Drive mirror: https://drive.google.com/drive/folders/1N2Qx2qkFgRtpbHitl2Vh6sLVYGgqkWwn

SoccerTrack v2 paper: https://arxiv.org/abs/2508.01802

GSR format/release warnings: https://github.com/AtomScott/SoccerTrack-v2/blob/main/docs/format-gsr.md

BAS format: https://github.com/AtomScott/SoccerTrack-v2/blob/main/docs/format-bas.md

Data corrections: https://github.com/AtomScott/SoccerTrack-v2/tree/main/data_corrections

FAANTRA paper: https://arxiv.org/abs/2504.12021

FAANTRA code: https://github.com/MohamadDalal/FAANTRA

SoccerNet 2026 report: https://arxiv.org/abs/2607.07320

SoccerNet 2026 challenge: https://www.soccer-net.org/challenges/2026

Ochin game-state detection paper: https://www.scitepress.org/PublishedPapers/2025/131611/

Beyond Pixels: https://arxiv.org/abs/2505.09455

Google Colab FAQ: https://research.google.com/colaboratory/faq.html

---

Final principle: the feasibility study passes only when the data path is reproducible, alignment is defensible, preprocessing is restartable, the tiny model can learn its small batch, and measured resource use supports a full-scale plan with reserve. Accuracy on the 10-minute pilot is not the research result.